

Workbench 真实未读与排序后端改造说明

1. 本次改造目标

围绕工作台仓库会话，补齐以下后端能力：

- 为每个用户的每个仓库会话记录 `lastReadAt`
- 会话列表排序统一以后端计算的 `latestMessageAt` 为准
- 未读统计不再依赖 `Notification.readAt`，而是基于会话消息时间与 `lastReadAt` 比较
- 会话列表返回未读数量、未读最高风险等级、风险分布、待审批状态
- 会话消息列表返回统一时间线字段，包含 `isUnread`、`riskLevel`、审批动作等
- 新增会话已读接口，支持显式标记某个仓库会话为已读

2. 代码改动清单

2.1 数据库

修改文件：

- `packages/database/prisma/schema.prisma`
- `packages/database/prisma/migrations/20260521220000_add_user_repository_conversation_state/migration.sql`

新增模型：

```
model UserRepositoryConversationState {
  userId      String
  repositoryId String
  lastReadAt   DateTime?
  lastViewedAt DateTime?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}
```

说明：

- 使用 `(userId, repositoryId)` 作为主键，表示“用户-仓库会话”唯一状态
- `lastReadAt` 用于真实未读统计
- `lastViewedAt` 预留给后续“浏览过但未读完”或埋点场景
- 在 `User`、`Repository` 上补了关联字段

2.2 Workbench 接口

修改文件：

- `apps/api/src/modules/workbench/workbench.service.ts`
- `apps/api/src/modules/workbench/workbench.controller.ts`
- `apps/api/src/modules/workbench/dto/read-conversation.dto.ts`

新增接口：

```
POST /workbench/chat/repositories/:id/read
```

请求体：

```
{
  "readAt": "2026-05-21T13:00:00.000Z",
  "upToMessageAt": "2026-05-21T13:05:00.000Z"
}
```

说明：

- `upToMessageAt` 优先级高于 `readAt`
- 两者都不传时，后端会默认以“当前仓库会话最新消息时间”作为已读时间
- 后端会做幂等保护，不会把 `lastReadAt` 回退到更早的时间

3. 会话聚合口径

3.1 会话消息来源

当前统一纳入会话的消息源：

- `Event`
- `Approval`

当前未纳入：

- `Notification` 作为投递记录保留，不再作为 workbench 会话真实未读主来源
- `Agent` 待确认动作暂未落库，因此本次只保留接口字段占位

3.2 会话时间规则

后端统一 message time 计算方式：

- `Event`: `occurredAt ?? createdAt`
- `Approval`: `reviewedAt ?? createdAt`

基于此：

- 会话列表排序：取仓库下所有会话消息的最大时间
- 会话列表 `latestMessageAt`：同上
- 会话详情时间线排序：按统一 message time 倒序

3.3 真实未读规则

真实未读定义：

- 当消息时间 `messageTime > lastReadAt` 时，视为未读
- 当 `lastReadAt` 为空时，该仓库会话内所有消息均视为未读

这意味着：

- 未读统计不再等价于 `Notification.readAt is null`
- 它反映的是“仓库会话是否已阅读到某个时间点”

4. 返回字段变化

4.1 GET /workbench/chat/repositories

每个仓库会话项新增或强化了这些字段：

- `lastReadAt`
- `latestMessageAt`
- `latestMessageType`
- `latestMessagePreview`
- `unreadCount`
- `unreadRiskLevel`
- `unreadRiskCounts`
- `highRiskCount`
- `hasPendingApproval`
- `pendingApprovalCount`
- `hasPendingAgentAction`
- `pendingAgentActionCount`
- `requiresAttention`

风险分布示例：

```
{
  "unreadRiskCounts": {
    "LOW": 2,
    "MEDIUM": 1,
    "HIGH": 3,
    "CRITICAL": 0
  }
}
```

4.2 GET /workbench/chat/repositories/:id/messages

接口返回从“纯数组”调整为对象：

```
{
  "conversation": {
    "repositoryId": "repo_xxx",
    "lastReadAt": "2026-05-21T13:05:00.000Z",
    "unreadCount": 6,
    "unreadRiskLevel": "HIGH",
    "unreadRiskCounts": {
      "LOW": 1,
      "MEDIUM": 2,
      "HIGH": 3,

```

```
    "CRITICAL": 0
  },
  "hasPendingApproval": true,
  "pendingApprovalCount": 2,
  "hasPendingAgentAction": false,
  "pendingAgentActionCount": 0
},
"messages": []
}
```

`messages` 中的单条消息新增或强化了这些字段：

- `riskLevel`
- `isUnread`
- `hasPendingApprovalAction`
- `hasPendingAgentAction`
- `approvalId`
- `approvalStatus`
- `actions`

5. 风险等级口径

5.1 Event 风险

优先使用该事件最新一条已完成 AI 分析的 `riskLevel`。

5.2 Approval 风险

优先使用其关联事件最新 AI 分析的 `riskLevel`。

如果缺失分析结果：

- `PENDING` 审批回退为 `HIGH`
- 非 `PENDING` 审批回退为 `MEDIUM`

这样前端可以直接根据消息级别做不同颜色的气泡，而不是继续从事件类型反推风险。

6. 与现有通知系统的关系

本次没有移除现有通知模块，也没有改动：

- `Notification.readAt`
- `/notifications/unread-count`
- `/notifications/:id/read`

原因是：

- 通知系统仍然表示“投递记录是否已处理”
- Workbench 会话系统表示“仓库消息会话阅读进度”

两者语义不同，后续前端应逐步切换到 workbench 聚合接口展示真实会话未读。

7. 当前已知边界

7.1 Agent 待确认动作

这次已经在接口中预留：

- `hasPendingAgentAction`
- `pendingAgentActionCount`

但当前仓库里还没有对应的 Agent 操作持久化模型，所以本次返回：

- `false`
- `0`

后续如果引入例如 `AgentRun` / `AgentActionRequest` 之类的模型，可以直接接入同一套会话聚合逻辑。

7.2 前端工作台暂未完全消费该聚合接口

当前前端 `DesktopWorkbench` 仍有一部分是自行拼接 `events + approvals + notifications`。

因此建议下一步前端改造：

- 仓库会话列表改为直接消费 `/workbench/chat/repositories`
- 仓库消息面板改为直接消费 `/workbench/chat/repositories/:id/messages`
- 打开会话或滚动到确认时机后，调用 `/workbench/chat/repositories/:id/read`

8. 验证情况

本次已完成的本地验证：

- `pnpm.cmd --filter @repo-pulse/database db:generate`
- `pnpm.cmd --filter @repo-pulse/database build`
- `pnpm.cmd --filter @repo-pulse/api typecheck`
- `pnpm.cmd --filter @repo-pulse/api build`

9. 建议的后续工作

建议按下面顺序继续推进：

1. 前端工作台切换到 `workbench` 聚合接口，统一未读和排序口径
2. 为进入会话、显式“标记已读”、或滚动到底部等动作定义最终已读策略
3. 设计 Agent 待确认动作实体表，并接入同一会话聚合
4. 如果仓库事件量继续增长，再把当前内存聚合逐步替换为数据库级聚合查询